SQL PRACTICE ASSIGNMENT — PART 2

Intermediate & Hard Focus: CTEs, Window Functions, HAVING, Joins

RetailAnalyticsDB | 64 Questions | Intermediate → Hard

Builds on: Part 1 (RetailAnalyticsDB — Departments, Employees, Customers, Categories, Suppliers, Products, Orders, OrderDetails). This part adds two more tables, `Payments` and `Returns`, to give the analytical and CTE/window-function questions richer, more realistic data to work with.

Topics emphasized: CTEs, Window / Analytical Functions, HAVING, GROUP BY, WHERE, all Join types, String Functions, Date Functions, plus CREATE TABLE / INSERT practice.

Tool: Microsoft SQL Server Management Studio (SSMS)

How to use: Run the setup script below (it assumes Part 1's database and data already exist). Then work through Intermediate (Q1–Q39) before Hard (Q40–Q64). A companion Solutions PDF has fully worked answers.

Part 1 — Setup Addendum: Payments & Returns

If you haven't already, run Part 1's full setup script first (creates `RetailAnalyticsDB` with 8 tables and sample data). Then run the script below to add two more tables that several questions in this part rely on.

1.1 Create Tables

```
USE RetailAnalyticsDB;
GO

CREATE TABLE Payments (
    PaymentID       INT IDENTITY(1,1) PRIMARY KEY,
    OrderID         INT NOT NULL REFERENCES Orders(OrderID),
    PaymentDate     DATE NOT NULL,
    Amount          DECIMAL(10,2) NOT NULL,
    PaymentMethod   VARCHAR(20) NOT NULL
);

CREATE TABLE Returns (
    ReturnID        INT IDENTITY(1,1) PRIMARY KEY,
    OrderDetailID   INT NOT NULL REFERENCES OrderDetails(OrderDetailID),
    ReturnDate      DATE NOT NULL,
    Reason          VARCHAR(100),
    RefundAmount    DECIMAL(10,2)
);
GO
```

1.2 Load Sample Data

```
INSERT INTO Payments (OrderID, PaymentDate, Amount, PaymentMethod) VALUES
(1,  '2023-01-05', 1997.00,  'UPI'),
(2,  '2023-01-12', 2492.50,  'Credit Card'),
(3,  '2023-01-20', 2347.10,  'Net Banking'),
(4,  '2023-02-02', 5749.00,  'Credit Card'),
```

```
(6,  '2023-02-28', 1898.00,  'UPI'),
(7,  '2023-03-10', 1256.00,  'Debit Card'),
(8,  '2023-03-15', 3448.15,  'UPI'),
(10, '2023-04-01', 2200.00,  'Credit Card'),
(11, '2023-04-11', 3798.10,  'Cash on Delivery'),
(12, '2023-04-18', 6298.00,  'Credit Card'),
(13, '2023-05-02', 3395.00,  'UPI'),
(15, '2023-05-25', 680.00,   'Debit Card'),
(16, '2023-06-05', 8998.20,  'Credit Card'),
(17, '2023-06-18', 4498.00,  'UPI'),
(18, '2023-07-01', 945.00,   'Net Banking'),
(20, '2023-08-02', 3298.00,  'Credit Card'),
(21, '2023-09-10', 4400.00,  'UPI'),
(22, '2023-10-05', 3295.00,  'Debit Card'),
(23, '2023-11-20', 2845.25,  'UPI'),
(24, '2024-01-08', 1884.00,  'Credit Card');

INSERT INTO Returns (OrderDetailID, ReturnDate, Reason, RefundAmount) VALUES
(8,  '2023-02-10', 'Wrong item received',   300.00),
(15, '2023-03-25', 'Defective product',    2549.15),
(22, '2023-04-25', 'Damaged in transit',   2799.00),
(37, '2023-09-20', 'Changed mind',          900.00),
(2,  '2023-01-12', 'Not compatible',        799.00);
GO
```

OrderDetailID reference: Row 8 = Order 4 / Notebook Pack of 5. Row 15 = Order 8 / Winter Jacket. Row 22 = Order 12 / Bookshelf. Row 37 = Order 21 / Almonds 1kg. Row 2 = Order 1 / USB-C Charger. (These follow the exact insertion order of OrderDetails from Part 1.)

Table	Key Columns	Notes
Payments	PaymentID (PK), OrderID (FK)	20 rows — one per delivered order
Returns	ReturnID (PK), OrderDetailID (FK)	5 rows — partial/full returns on specific line items

Part 2 — Practice Questions

WHERE Clause

- Q1Retrieve all payments made via `'UPI'` or `'Credit Card'` with an `Amount` greater than 2000.
- Q2Retrieve all returns whose `Reason` contains the word `'damaged'` or `'defective'` (case-insensitive).

Aggregate Functions

- Q3Find the total refund amount issued across all returns.
- Q4In a single query, find the total, average, and count of all payment amounts.
- Q5Find the earliest and latest `PaymentDate` recorded.

GROUP BY

- Q6Find the total payment amount collected, grouped by `PaymentMethod`.
- Q7Find the number of returns, grouped by `Reason`.
- Q8Find total revenue from `OrderDetails`, grouped by **both** `CategoryID` and the year of `OrderDate`.

HAVING

- Q9Find payment methods that have collected **more than 5000** in total.
- Q10Find categories where total revenue **exceeds 3000**.

- Q11Find customers who have placed **more than 2 orders AND** spent **more than 3000** in total (two aggregate conditions in one HAVING).
- Q12Find products that have been returned **more than once**.
- Q13Find months where the **average order value** is above 2000.

Joins

- Q14**INNER JOIN:** List every payment along with the customer's name and the order date.
- Q15**INNER JOIN:** List every return along with the product name and the original order date.
- Q16**LEFT JOIN:** List every delivered order with its payment details, including any delivered order that has no payment recorded.
- Q17**LEFT JOIN:** List every order-detail line item with its return info (if any), including line items that were never returned.
- Q18**RIGHT JOIN:** List every customer and their payments (via Orders), ensuring customers with zero payments still appear.
- Q19**CROSS JOIN:** Generate every combination of distinct `PaymentMethod` × `CategoryName` (a coverage matrix).
- Q20**FULL OUTER JOIN:** Combine `OrderDetails` and `Returns` so that line items with no return, and any return without a matching line item, both appear.

String Functions

- Q21Display each `PaymentMethod` in lowercase, prefixed with the text `'method: '`, using `CONCAT()` and `LOWER()`.
- Q22Extract just the domain part (after the `@`) of every customer's email using `SUBSTRING()` and `CHARINDEX()`.
- Q23Replace every space in `ProductName` with an underscore using `REPLACE()`.
- Q24Trim any leading/trailing spaces from the `Reason` column in `Returns` using `TRIM()`.

Date Functions

- Q25Find how many days after the `OrderDate` each payment was made, using `DATEDIFF()`.
- Q26Display every `PaymentDate` formatted as `'DD-Mon-YYYY'` using `FORMAT()`.
- Q27Find all returns that happened within 30 days of the original order date.

CTE (Common Table Expressions)

- Q28Write a CTE that computes total payments per customer, then select customers whose total payments exceed 5000.
- Q29Write a CTE to compute total refund amount per product (via `Returns` → `OrderDetails`), then join it back to `Products` to show `ProductName` and `TotalRefunded`.
- Q30Write a CTE that computes monthly payment totals, then select the month with the **lowest** total.
- Q31Write **two** CTEs — one for total revenue per customer and one for total refunds per customer — then join them to compute each customer's `NetRevenue`.
- Q32Use a CTE to find each customer's average order value, then list customers whose average order value is above the **overall** average order value.

Window / Analytical Functions

- Q33Use `SUM() OVER (PARTITION BY CustomerID)` to show each order's revenue next to that customer's **total** revenue across all their orders — without collapsing rows with `GROUP BY`.
- Q34Use `ROW_NUMBER()` to isolate each customer's **most recent** order (exactly one row per customer).
- Q35Use `RANK()` to rank payment methods by total amount collected.

- Q36Use `LAG()` to compare each payment's `Amount` with the same customer's previous payment, and flag whether it `'Increased'`, `'Decreased'`, or is their `'First Payment'`.
- Q37Use `PERCENT_RANK()` to show each product's relative price standing within its category.
- Q38Use `COUNT() OVER (PARTITION BY ...)` to show, next to every return row, the total number of times that same product has been returned.

CREATE TABLE + INSERT

- Q39Design and create a table `Cart` (`CartID` PK, `CustomerID` FK, `ProductID` FK, `DateAdded` defaulting to today, `Quantity` defaulting to 1) with appropriate constraints, then insert 3 sample rows referencing existing customers and products.

- Q40Using a CTE + window function, find the **top 2 highest payments** made by each customer.
- Q41Using `HAVING` with a subquery, find products whose total return count is greater than the **average** return count among all products that have ever been returned.
- Q42Using CTEs, find any customers whose **net revenue** (total payments minus total refunds) is less than or equal to zero.
- Q43Using a window function, calculate the **running total** of refunds issued over time, ordered by `ReturnDate`.
- Q44Using `LAG()`, find the number of days between **consecutive orders** placed by the same customer, and flag any gap greater than 60 days as `'Inactive Period'`.
- Q45Write a **recursive CTE** that generates a full calendar "date spine" of every date between the earliest and latest `OrderDate`, then `LEFT JOIN` it to `Orders` to show which days had zero orders.
- Q46Using a CTE + `RANK() ... PARTITION BY`, find the best-selling product (by revenue) **within each payment method** (i.e. join through `Payments` to see which method was used most for each top seller).
- Q47Using `SUM() OVER ()`, find what **percentage** of total company revenue each category contributes.
- Q48Using `CUME_DIST()`, identify which products fall in the **top 10%** by price.
- Q49Using a CTE, find customers who have placed orders in **at least 3 different calendar months** (a loyalty/engagement pattern).
- Q50Using window functions, compute the **month-over-month percentage growth** in total revenue.
- Q51Using a window function with a frame clause, find the **3 consecutive months** with the highest combined revenue (a sliding window).
- Q52Using `HAVING` and a CTE, find categories where the **return rate** (returned units ÷ total units sold) exceeds 10%.
- Q53Using a window function, flag each row in `Returns` as that product's `'First Return'` or `'Repeat Return'`, based on `ReturnDate` order.
- Q54Using a self-join or window function, find pairs of orders from the **same customer** placed within 7 days of each other.
- Q55Using `FULL OUTER JOIN` and `COALESCE()`, build a reconciliation report comparing each order's total amount (from `OrderDetails`) against the amount actually paid (from `Payments`), flagging any mismatch — including orders with **no** payment row at all.
- Q56Using a CTE and `NTILE(3)`, classify customers into `'Gold'`, `'Silver'`, and `'Bronze'` tiers based on total spend.
- Q57Using `DATEPART()` with `GROUP BY` and `HAVING`, find which day of the week generates the highest **average** order value.
- Q58Using window functions, compare each product's revenue rank in one month against its rank in another month (pick any two months present in the data) and show the rank change.
- Q59Using a CTE + JOIN + HAVING, find suppliers whose products have generated more than 5000 in total revenue.
- Q60Using `STRING_AGG()`, produce one row per order listing all of its product names as a single comma-separated string.
- Q61Using a window function, detect potential **duplicate / double-charge** payments — two payments for the same customer, same date, same amount.

- Q62Design and create a table `OrderStatusHistory` (`HistoryID` PK, `OrderID` FK, `OldStatus`, `NewStatus`, `ChangedDate`). Insert at least 4 rows reflecting plausible status transitions for existing orders, then use a window function to compute, for each order, how long it stayed in its previous status before changing.
- Q63Using a CTE + window functions, compute each customer's rank by total spend **overall** and, side by side, their rank by total spend **within their own State**.
- Q64A dashboard needs "revenue per category per quarter" with each category as a row and each quarter as a column. Using `DATEPART(QUARTER, ...)`, a CTE, and conditional aggregation (or `PIVOT`), reshape the data accordingly.

End of Part 2 — 64 questions total (39 Intermediate · 25 Hard). See the companion Solutions PDF (Part 2) for fully worked T-SQL answers.